VR Technical Training Bay Assessment

Aonix Practical Round Assessment Handoff • Candidate: **Vishnu Babu**

UNITY 6 (6000.3.20F1) | META QUEST STANDALONE | 86.6 FPS • ZERO-GC

ALL FOUR MANDATORY DELIVERABLES

1. Unity Project & Source Code
Deliverable 01

Clean project preserving package manifests & settings. Excludes Library, Temp, Logs, and obj folders.

GitHub Repo | Clean ZIP (Drive)

`VCS: plastic://vishnu08.unity/repos/VR_task%2fVR_task`

2. Installable Meta Quest APK
Deliverable 02

Compiled Android ARM64 IL2CPP build for Meta Quest 2 / 3. Boots directly into training bay.

Download APK (Google Drive)

3. 3–5 Minute Video Demonstration
Deliverable 03

Captures full user journey, Type-A rejection, distance recovery, restart mechanism, and scene inspection.

Watch Video (Google Drive)

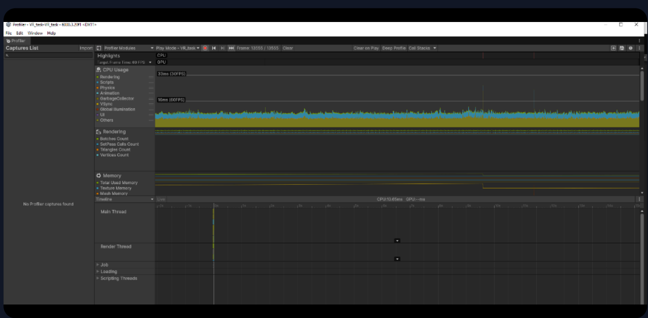
4. Comprehensive README.md
Deliverable 04

Full 8-point checklist documentation: versions, controls, architecture, licences, and development notes.

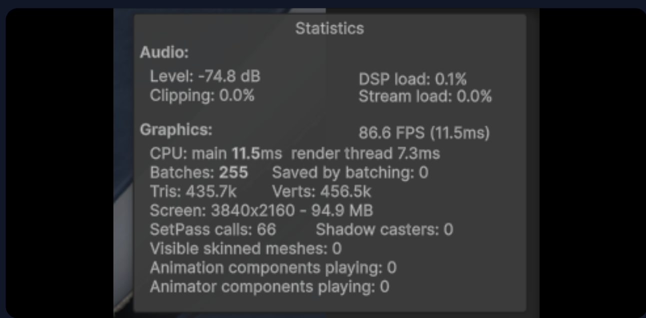
View README on GitHub

STANDALONE QUEST PERFORMANCE & FRAME BUDGET PROOF

86.6 FPS	**11.5 ms**	**66**	**0 B**
STABLE FPS (BUDGET: 72HZ)	CPU MAIN FRAME TIME	SETPASS CALLS (< 100 LIMIT)	STEADY-STATE GC ALLOC



Profiler: Frame Time < 12ms, 0ms Garbage Collector Spikes



Game View Stats: 86.6 FPS, 255 Batches, 66 SetPass Calls

UNITY VERSION CONTROL & ENGINEERING EVOLUTION

Unity Version Control (Plastic SCM): plastic://vishnu08.unity/repos/VR_task%2fVR_task • **21 Changesets (cs:0 – cs:20)** on branch /main • Owner: vishnubabu1108@gmail.com

Phase 1 (cs:0 – cs:5) — Project Baseline & Meta Interaction SDK
Project configured on Unity 6 (6000.3.20f1) URP. Configured OpenXR with Meta Quest feature group, XR Interaction SDK, and world-space ray pointer UI controls.

Phase 2 (cs:6 – cs:9) — Interaction Controls & Distance Safety
Built ray pointer interaction system, Touch controller bindings, and added DistanceRecovery.cs to prevent cells from falling through floor bounds.

Phase 3 (cs:10 – cs:11) — Calibration Dial Precision & Constraints
Engineered CalibrationDial.cs with 70°–85° target tolerance, 1.0s continuous hold requirement, and rotation transformer constraints.

Phase 4 (cs:12 – cs:15) — Power Cell Sockets & Type-A Rejection
Implemented CellSocket.cs with Type-B Hex validation and Type-A rejection mechanics (haptic buzz, audio error, red LED). Added scale compensation.

Phase 5 (cs:16 – cs:20) — Automation, Zero-GC Optimization & Standalone APK
Blast doors coordinator, video screen render texture, RestartButtonHandler.cs dial anchoring, shader multiview fixes, and 86.6 FPS standalone Quest build.

README CHECKLIST & TECHNICAL ARCHITECTURE SUMMARY

✓ 1. Baseline & Versions
Unity 6 (6000.3.20f1) URP • Scripting: IL2CPP / .NET Standard 2.1 • Meta XR Core & Interaction SDK v72+ • OpenXR 1.14+.

✓ 2. Controls & Flow
Laser pointer with Trigger/A to click UI buttons. Grip button to grasp power cells and turn dials right into 70°–85°. Complete cycle → Restart cleanly resets scene.

✓ 3. Architecture & State
Decoupled state machine (AssessmentManager) driven by UnityEvents. CellSocket handles scale-compensated parenting. RestartButtonHandler resets dials via reflection.

✓ 4. Device & Runtime Verified
Verified on Meta Quest via Meta Horizon Link, XR Simulator, and Standalone ARM64 APK. Confirmed 72Hz+ locked, 0 B runtime GC alloc.

✓ 5. 3 Concrete Optimizations
1. Zero-GC in core loop (NonAlloc physics). 2. MaterialPropertyBlock for dynamic glows preserving SRP batching. 3. Kinematic locking on console dials.

✓ 6. Asset Credits & Licences
Kenney Prototype Kit (CC0 Public Domain). Mixkit Audio (Free Commercial License). Meta XR Interaction SDK (Meta Technologies License).

✓ 7. Known Issues & Next Steps
Add Meta synthetic hand poses (HandGrabPose), upgrade procedural rumble to pre-authored .haptic audio clips, bake mixed lightmaps for Quest 2.

✓ 8. Development Notes
IDE tooling and code completion assistants were used for boilerplate syntax and math formulas. All architecture, interactions, physics, and state flows were engineered directly in Unity.